

Técnico de patrones de diseño

Para identificar el patrón de priorización que se necesita tomaremos las matrices de: **Urgencia, Priorización y de Impacto, Esfuerzo, Priorización;**

Al analizar las matrices se puede notar que las funcionalidades más críticas del proyecto no son independientes entre sí. Cuando se calcula el puntaje de prioridad, se identifican qué eventos del sistema tienen alto impacto con esfuerzo moderado, cuáles deben estar disponibles desde las primeras versiones, y a cuántos actores afectan simultáneamente.

Lo que emerge de ese análisis es un patrón claro en la naturaleza del sistema: **un evento ocurre en un punto y múltiples partes deben reaccionar automáticamente.** Las funcionalidades prioritarias no son acciones aisladas de un solo módulo, sino eventos que desencadenan respuestas en cadena hacia múltiples actores del sistema al mismo tiempo.

Para este proyecto usaremos el patrón de diseño de Observer.

Observer está diseñado para resolver situaciones donde un cambio en un objeto necesita reflejarse de manera automática en otros objetos, sin que el objeto que cambia tenga que conocer quiénes son ni cuántos son los que deben reaccionar. Esto significa que Observer fue diseñado para problemas de: **acoplamiento fuerte, consistencia entre objetos relacionados y de escalabilidad de reacciones.**

Por lo tanto, las soluciones reutilizables que brinda Observer son:

Desacoplamiento entre módulos: El módulo de citas no necesita saber que existe un módulo de notificaciones, uno de historial clínico y uno de agenda médica. Solo registra el cambio de estado y Observer se encarga de avisar a todos. Si en el futuro se agrega un módulo de farmacia o laboratorio, el módulo de citas no se toca.

Sincronización automática entre actores: Cuando ocurre un evento en el sistema, paciente, médico y administrativo se actualizan en el mismo instante sin que ninguno dependa del otro. No hay riesgo de que un actor quede desactualizado porque el aviso se disparó manualmente y alguien olvidó incluirlo.

Gestión de estados de la cita: Cuando una cita pasa de pendiente a confirmada, o de confirmada a cancelada, Observer dispara automáticamente todas las reacciones que ese cambio implica en cada módulo del sistema, sin necesidad de lógica adicional en el módulo principal.

Escalabilidad sin modificar código existente: Si el proyecto crece y aparece un nuevo actor y/o módulo que necesita reaccionar ante los mismos eventos, simplemente se registra como un nuevo observador. El código que ya funciona no se modifica, lo que reduce el riesgo de introducir errores en funcionalidades ya probadas.

Trazabilidad y auditoría: Cada observador puede registrar independientemente los eventos que recibe, lo que permite construir un historial de cambios del sistema sin que esa responsabilidad recaiga sobre el módulo central. El módulo de historial clínico es simplemente un observador más que escucha y registra.

Integración con la pasarela de pago: Cuando un pago se confirma, el cambio de estado de falso a verdadero notifica automáticamente al paciente, al médico y al sistema de facturación como observadores independientes del mismo evento, exactamente como lo describió el profesor en clase.

